

Lab 06: Introduction to Classes, Objects, Constructor and Encapsulation

Learning Objectives

By the end of this lab, you will understand:

- Understand the concept of classes and objects in Java.
- Create and use constructors.
- Apply method overloading.
- Understand and implement encapsulation.
- Relate Java concepts to real-life examples.

Classes and Objects: In real life, everything around us can be viewed as an object. For example:

- A car has properties like color, brand, and speed.
- A student has properties like name, roll number, and semester.
- A mobile phone has properties like model, battery percentage, and camera.

In Java:

- A class is a blueprint or design.
- An object is the real thing created from that blueprint.

For example:

- "Car" is a class.
- "My Red Toyota" is an object.

Real-Life Application

- Student Management Systems
- Banking Applications
- Hospital Management Systems
- Robotics Systems
- Online Shopping Platforms

Code Example

```
// Simple Car class (blueprint)
class Car {
    String brand;
    int speed;

    void showDetails() {
```

```
        System.out.println(brand + ", speed: " + speed);
    }
}
// Main class
public class Main {
    public static void main(String[] args) {
        // Creating objects
        Car car1 = new Car();
        car1.brand = "Toyota";
        car1.speed = 120;

        Car car2 = new Car();
        car2.brand = "Honda";
        car2.speed = 100;

        car1.showDetails(); // Toyota, speed: 120
        car2.showDetails(); // Honda , speed: 100
    }
}
```

Notes

- A class is a template.
- An object is created using the new keyword.
- Variables inside a class are called attributes or data members.
- Functions inside a class are called methods.

Task: Create a Phone class with brand and price. Create two phone objects and display them.

Constructor: A constructor is a special method that is automatically called when an object is created.

The constructor:

- Has the same name as the class.
- Does not have a return type.
- Is mainly used to initialize object values.

Without constructors, we often need to assign values manually after creating objects.

Real-Life Application

- Creating a student with a predefined name and roll number.
- Creating a bank account with an account title and balance.
- Creating a robot object with initial speed and direction.

Code Example (Parametrized Constructor).

```
class Student {
    String name;
    int rollNo;

    Student(String n, int r) {
        name = n;
        rollNo = r;
    }

    void display() {
        System.out.println("Name: " + name);
        System.out.println("Roll No: " + rollNo);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("Sara", 102);

        s1.display();
    }
}
```

Notes

- Constructors run automatically when an object is created.
- Constructors help reduce repeated code.
- Constructors are mainly used for initializing data.
- A class can have more than one constructor.

Task: Add default constructor in the above example and re-run the code so that it should print default values when not defined.

Constructor Overloading: Constructor overloading means creating multiple constructors in the same class with different parameters. This allows us to create objects in different ways depending on the available information.

Java decides which constructor to call based on:

- Number of parameters
- Type of parameters
- Order of parameters

Real-Life Application

- A student object may be created with only a name or with both name and roll number.
- A car object may be created with only brand or with brand and model.
- A robot object may be created with default speed or custom speed.
- A bank account may be created with or without an initial balance.

Code Example:

```
class Student {
    String name;
    int rollNo;

    Student() {
        name = "Unknown";
        rollNo = 0;
    }

    Student(String n) {
        name = n;
        rollNo = 0;
    }

    Student(String n, int r) {
        name = n;
        rollNo = r;
    }

    void display() {
        System.out.println("Name: " + name);
        System.out.println("Roll No: " + rollNo);
        System.out.println();
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student();
        Student s2 = new Student("Ali");
        Student s3 = new Student("Sara", 105);

        s1.display();
        s2.display();
        s3.display();
    }
}
```

Notes

- A class can have multiple constructors.
- Constructors must have different parameter lists.
- Constructor overloading improves flexibility.
- Constructor overloading is an example of compile-time polymorphism.

Task: Create a class of a robot in which its speed object may be created with default speed or custom speed.

Method Overloading: Method overloading means creating multiple methods with the same name but different parameters. This makes the code easier to read because similar tasks can use the same method name.

Java decides which method to call based on:

- Number of parameters
- Type of parameters
- Order of parameters

Real-Life Application

- Calculators
- Banking Systems
- Robotics Control Systems
- Geometry Programs

For example, a calculator may have:

- add(int a, int b)
- add(int a, int b, int c)
- add(double a, double b)

Code Example:

```
class Calculator {  
  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
  
    double add(double a, double b) {  
        return a + b;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {
```

```
Calculator c = new Calculator();

System.out.println(c.add(5, 3));
System.out.println(c.add(5, 3, 2));
System.out.println(c.add(2.5, 1.5));
}
}
```

Notes

- Method overloading improves readability.
- Overloaded methods must have different parameter lists.
- Changing only the return type is not enough for overloading.
- Overloading is an example of compile-time polymorphism.

Task: Create a class named Printer with overloaded methods named print() for:

- Printing one integer
- Printing two integers
- Printing one string

Call all methods from the main function.

Passing Objects to Methods: In Java, an object can also be passed as an argument to a method. This allows one object to use information from another object.

Passing objects is useful when:

- One class needs to work with another class
- We want to avoid repeating code
- We want to compare or display information from multiple objects

For example, a robot object may be passed to another method to display its status.

Code Example:

```
class Student {
    String name;
    int rollNo;

    Student(String n, int r) {
        name = n;
        rollNo = r;
    }
}

class Display {
    void showStudent(Student s) {
        System.out.println("Name: " + s.name);
        System.out.println("Roll No: " + s.rollNo);
    }
}
```

```

}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("Ali", 101);

        Display d = new Display();
        d.showStudent(s1);
    }
}

```

Task Implement the following applications

- Modify object values
- Perform calculations using object attributes
- Compare objects

This keyword: The this keyword is used to refer to the current object of the class. It is commonly used when constructor parameters and class variables have the same name.

Without this, Java becomes confused between:

- Class variables
- Constructor or method parameters

The this keyword helps clearly identify the class variable.

Code Example

```

class Student {
    String name;
    int rollNo;

    Student(String name, int rollNo) {
        this.name = name;
        this.rollNo = rollNo;
    }

    void display() {
        System.out.println("Name: " + name);
        System.out.println("Roll No: " + rollNo);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("Ahmed", 103);

        s1.display();
    }
}

```

Task: Use this chain in above code to enable different ways of defining constructors in the main code.

Encapsulation: Encapsulation means hiding data and allowing controlled access through methods. Normally, class variables should not be directly accessed from outside the class. Instead, we use:

- Private variables
- Public getter methods
- Public setter methods

This improves:

- Security
- Data protection
- Code organization

Real-Life Application

- ATM Systems
- Bank Accounts
- Employee Records
- Student Portals
- Robotics Sensor Data Handling

For example: A bank account balance should not be directly changed from outside the class. Instead, it should only be updated through deposit and withdraw methods.

Code Example:

```
class BankAccount {
    private double balance;

    void setBalance(double b) {
        balance = b;
    }

    double getBalance() {
        return balance;
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount();
        acc.setBalance(5000);

        System.out.println("Balance: " + acc.getBalance());
    }
}
```


Notes

- Private variables cannot be accessed directly outside the class.
- Getter methods are used to read values.
- Setter methods are used to update values.
- Encapsulation protects data from unwanted access.

Task: Apply encapsulation and then solve the logical and security issue in the following code.

```
class BankAccount {  
    public double balance;  
}  
class Main {  
    public static void main(String[] args) {  
        BankAccount acc = new BankAccount();  
        acc.balance = -5000;  
        System.out.println(acc.balance);  
    }  
}
```

Static Members: Static members belong to the class rather than individual objects. They can be:

- **Static variables**
- **Static methods**

A static variable is shared by all objects of the class. A static method can be called without creating an object.

Real-Life Application

- Counting total students in a class
- Counting total robots created in a robotics system
- Storing a common university name for all students
- Utility methods such as mathematical calculations

Code Example

```
class Student {  
    String name;  
    static String university = "ABC University";  
    static int count = 0;  
  
    Student(String name) {  
        this.name = name;  
        count++;  
    }  
}
```

```
static void displayCount() {  
    System.out.println("Total Students: " + count);  
}  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Student s1 = new Student("Ali");  
        Student s2 = new Student("Sara");  
  
        System.out.println(s1.name + " studies at " + Student.university);  
        System.out.println(s2.name + " studies at " + Student.university);  
  
        Student.displayCount();  
    }  
}
```

Importance

- Static variables save memory because only one copy is shared.
- Static methods can be called without creating objects.
- Static members are useful for common information shared by all objects.

Task: Create a class named Robot with:

- A static variable named labName
- A static variable to count how many robot objects are created
- A static method to display total robots

Final Keyword for Variables: The final keyword is used to make a variable constant. Once a final variable is assigned a value, it cannot be changed.

Real-Life Application

- Storing fixed university names
- Defining mathematical constants such as pi
- Storing robot maximum speed limits
- Defining fixed tax percentages

Importance

- Final variables protect important values from being changed.
- They make the code safer and easier to understand.
- They are useful for fixed values that should remain constant.

Code Example:

```
class Robot {
    final int MAX_SPEED = 100;

    void display() {
        System.out.println("Maximum Speed: " + MAX_SPEED);
    }
}

public class Main {
    public static void main(String[] args) {
        Robot r1 = new Robot();
        r1.display();
    }
}
```

Task: Create a class named University with:

- A final variable named country with value Pakistan
- A method to display the country name

Exercises

Task 1

Create a class named Smartphone with:

- Attributes: company, cpu type.
- Method: displayCarInfo()

Create two smartphone objects and display their information.

Task 2

Create a class named Book with:

- Attributes: title, author, and price
- A constructor to initialize all values
- A method to display book information

Create two book objects and display their details.

Task 3

Create a class named Calculator and overload a method named multiply() for:

- Two integers
- Three integers
- Two double values

Call all three methods from the main function.

Task 4

Create a class named Employee using encapsulation:

- Private variables: salary and department
- Setter methods: setSalary() and setDepartment()
- Getter methods: getSalary() and getDepartment()

Create an employee object and display both values using getter methods.

Task 5

Create a class named Student with constructor overloading:

- One constructor with no parameters
- One constructor with only name
- One constructor with name and roll number

Display the data of all objects.

Task 6

Create a class named Robot with:

- Attributes: name and speed
- Constructor using the this keyword
- Method: displayRobotInfo()

Create two robot objects with different values and display their information.

Mini Project

Project Title

Student Management System

Problem Statement

Create a small Student Management System using the concepts covered in this lab.

The system should:

- Store student name, roll number, and semester
- Use a constructor to initialize student data
- Use constructor overloading
- Use encapsulation for CGPA
- Use a static variable to count total students
- Use a final variable for university name
- Display student information

Expected Output

Name: Unknown

Roll No: 0

Semester: 1

CGPA: 3.1

University: ABC University

Name: Ali

Roll No: 101

Semester: 3

CGPA: 3.7

University: ABC University

Name: Sara

Roll No: 102

Semester: 5

CGPA: 3.9

University: ABC University

Total Students: 3